Code Generation

* The final phase in compiler model is the code generator. It takes as input an intermediate representation of the source pgm. and produces as output an equivalent target pgm.

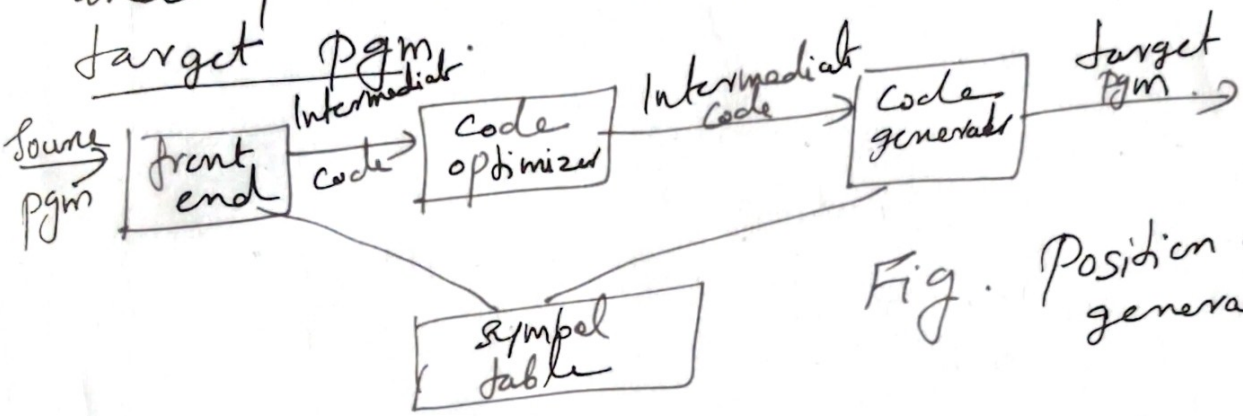

Fig. Position of code generator.

Issues in the design of a code Generator

(1) Input to the code generator

* The input to the code generator consists of the intermediate representation of the source pgm produced by the front end, together with information in the symbol table.
* There are several choices for the intermediate language including postfix notation, three address representations, graphical representations like syntax trees and dags.

Target Programs

* The output of the code generator is the target pgm. The output may take a variety of forms, absolute machine language, relocatable m/c language and assembly language.

* Producing an absolute machine language program as output has the advantage that it can be placed in a fixed location in memory and immediately executed.

START 100

* Producing a relocatable machine language pgm. as output allows subpgms. to be compiled separately. A set of relocatable object modules can be linked together and loaded for execution by a linking loader. Hence a great deal of flexibility in being able to compile subroutines separately and to call other previously compiled pgms from an object module.

* Producing an assembly language pgm. as output makes the process of code generation easier.

Memory management

* Mapping names in the source pgm. to addresses of data objects in run-time memory is done cooperatively by the front end and the code generator.
* Symbol table entries were created based on ~~as the~~ declarations ~~in a~~ statements. The type in a declaration determines the ~~the~~ amount of storage needed for the declared name.

Instruction selection

* The nature of the instruction set of the target machine determines the difficulty of instruction selection.

Consider $X = Y + Z$, can be translated in to the code sequence.

```
MOV Y, R0
ADD Z, R0
MOV R0, X
```

But this kind of statement by statement code generation often produces poor code.

Consider, $a = b + c$
$d = a + e$

would be translated in to

```
MOV 6, R0
ADD c, R0
MOV R0, a
MOV a, R0      ✓ redundant.
ADD e, R0
MOV R0, d
```

Here the 4th stat. is redundant. The quality of the generated code is determined by its speed and size.

* A target machine with a rich instruction set may provide several ways of implementing a given operation.
* If the target machine has an increment instruction (INC), then the three address statement a = a+1 may be implemented more efficiently by the single instruction INC a, rather than

```
MOV a, R0
ADD #1, R0
MOV R0, a
```

Register allocation

* Instructions involving register operands are usually shorter and faster than those

involving operands in memory. Therefore, efficient utilization of registers is particularly important in generating good code.

Consider,

$t = a+b$
$t = t*c$
$t = t/d$.

Then,

```
L R1, a
A R1, b
M R0, c
D R0, d
ST R1, t
```

Choice of evaluation order.

* The order in which computations are performed can affect the efficiency of the target code.
* Some computation order require fewer registers to hold intermediate results than others.

Consider,

$a = b+c$
$x = y+z$
$d = a+e$

$a = b+c$
$d = a+e$
$x = y+z$.